

机器学习大作业一实验报告

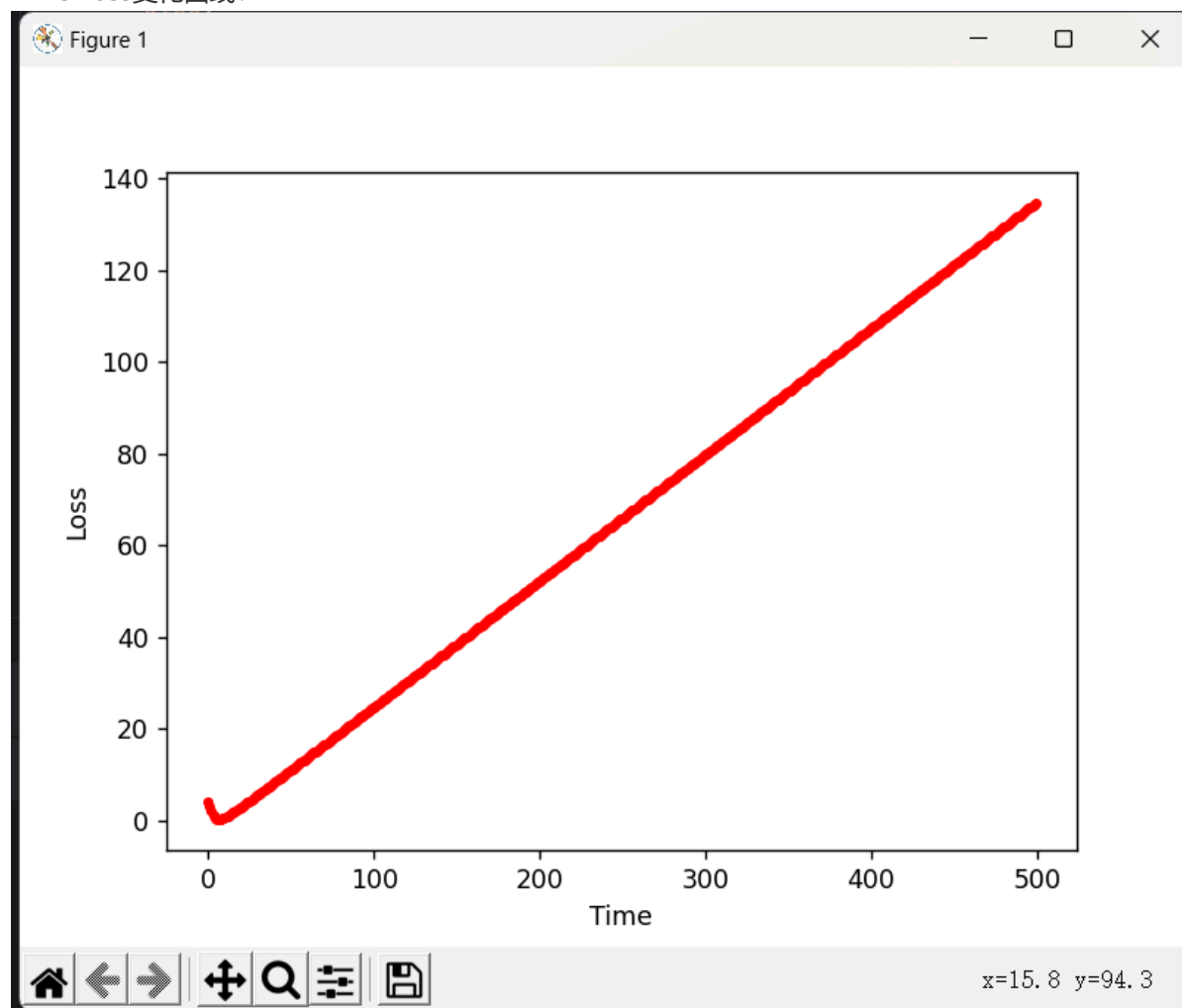
线性分类模型

Hinge Loss 损失函数:

函数实现:

```
3 usages
def Cost(X_normalized, Y, index, w, b): # 单个数据点的损失
    if index < 0 or index >= len(X_normalized):
        raise ValueError("Index is out of bounds.")
    linear_com = np.dot(w, X_normalized.iloc[index].values) + b
    # p = sigmoid(linear_com)
    # p = p.astype('float')
    # cost = - (Y.iloc[index].values * np.log(p) + (1 - Y.iloc[index].values) *
    cost = max(0, 1 - Y.iloc[index].values * linear_com)
    return cost
```

Time-Loss变化曲线:



正确率: 正确率: 0.6520210896309314

运行时间: 执行时间: 109.66996836662292

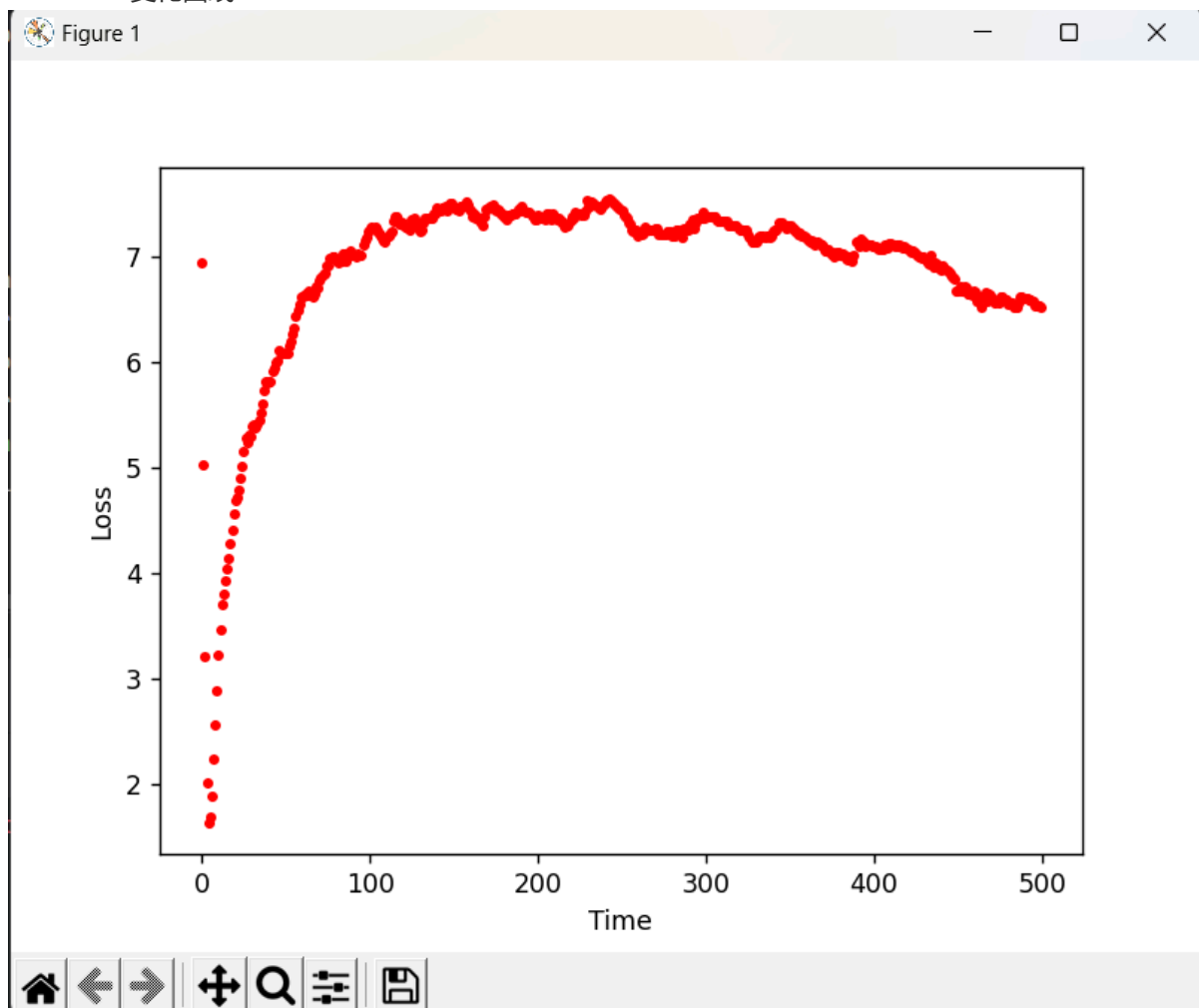
Cross-Entropy Loss 损失函数:

函数实现:

```
def sigmoid(x):  
    x=np.array(x,dtype=np.float64)  
    return 1 / (1 + np.exp(-x))
```

```
3 usages  
def Cost(X_normalized, Y, index, w, b): # 单个数据点的损失  
    if index < 0 or index >= len(X_normalized):  
        raise ValueError("Index is out of bounds.")  
    linear_com = np.dot(w, X_normalized.iloc[index].values) + b  
    p = sigmoid(linear_com)  
    p = p.astype('float')  
    cost = - (Y.iloc[index].values * np.log(p) + (1 - Y.iloc[index].values) * np.log(1 - p)) # Cross-Entropy L  
    return cost
```

Time-Loss变化曲线:



分类准确率: 正确率: 0.9015817223198594

分类所需时间: 执行时间: 94.34867215156555

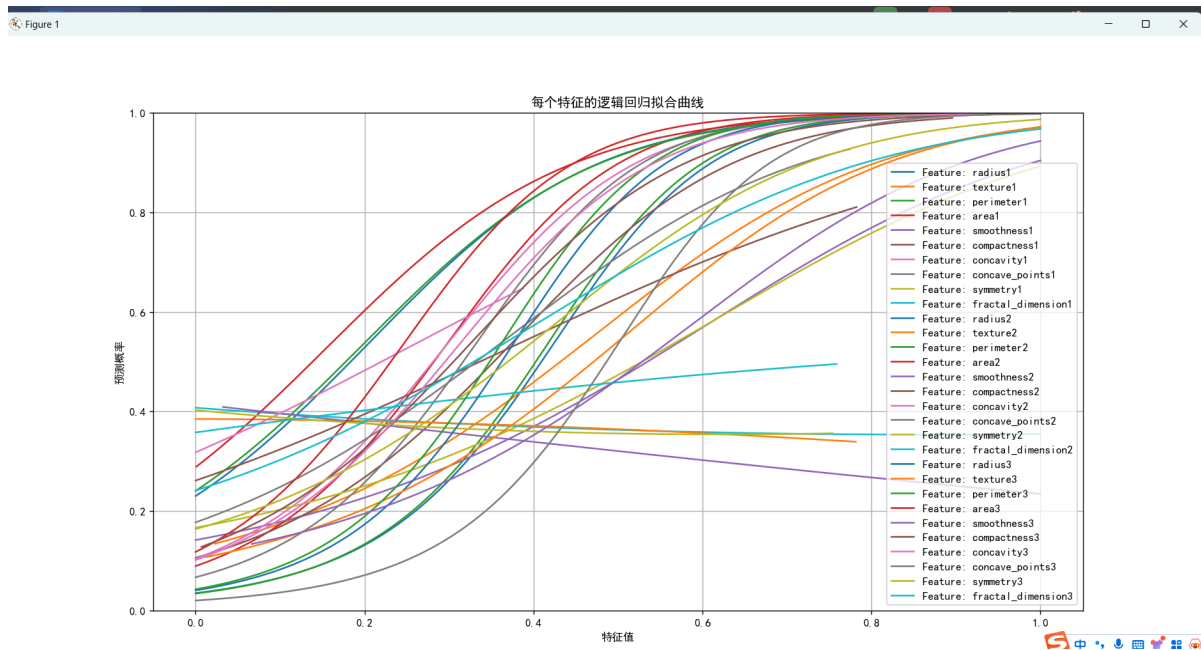
分析与对比

在使用线性分类器进行分类任务时, 使用Cross-Entropy Loss损失函数分类的准确率远高于使用 Hinge Loss损失函数时, 运行时间也更短。因此, 在线性分类器中, Cross-Entropy Loss损失函数的性能更好。

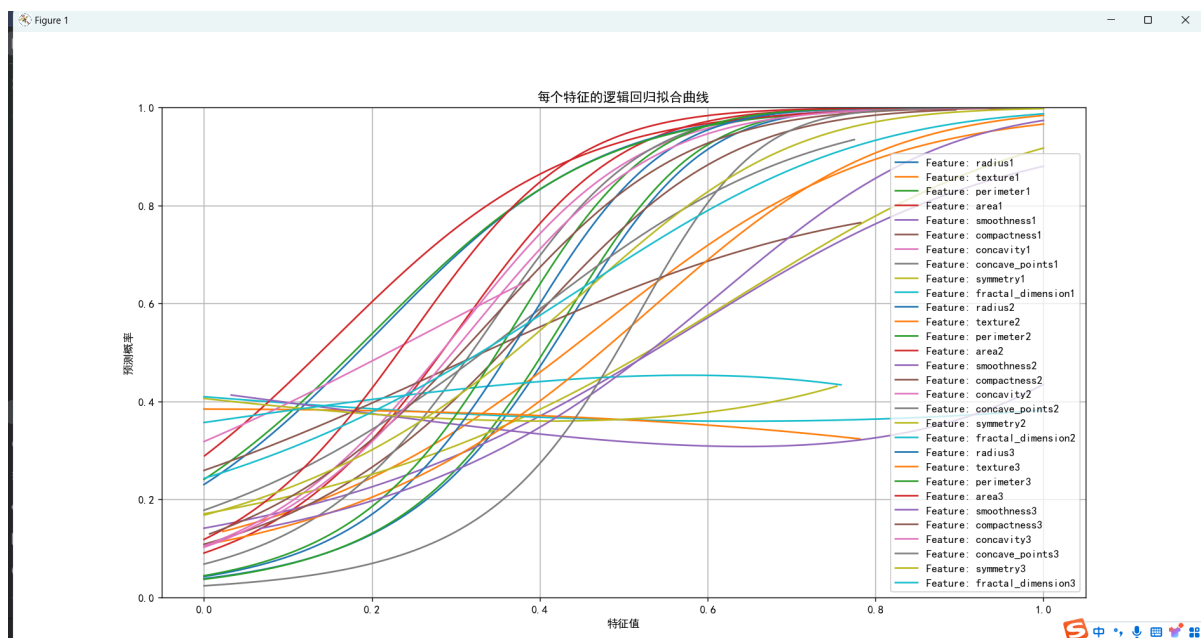
多项式分类器

分别训练不同特征, 得到拟合曲线

degree = 2:



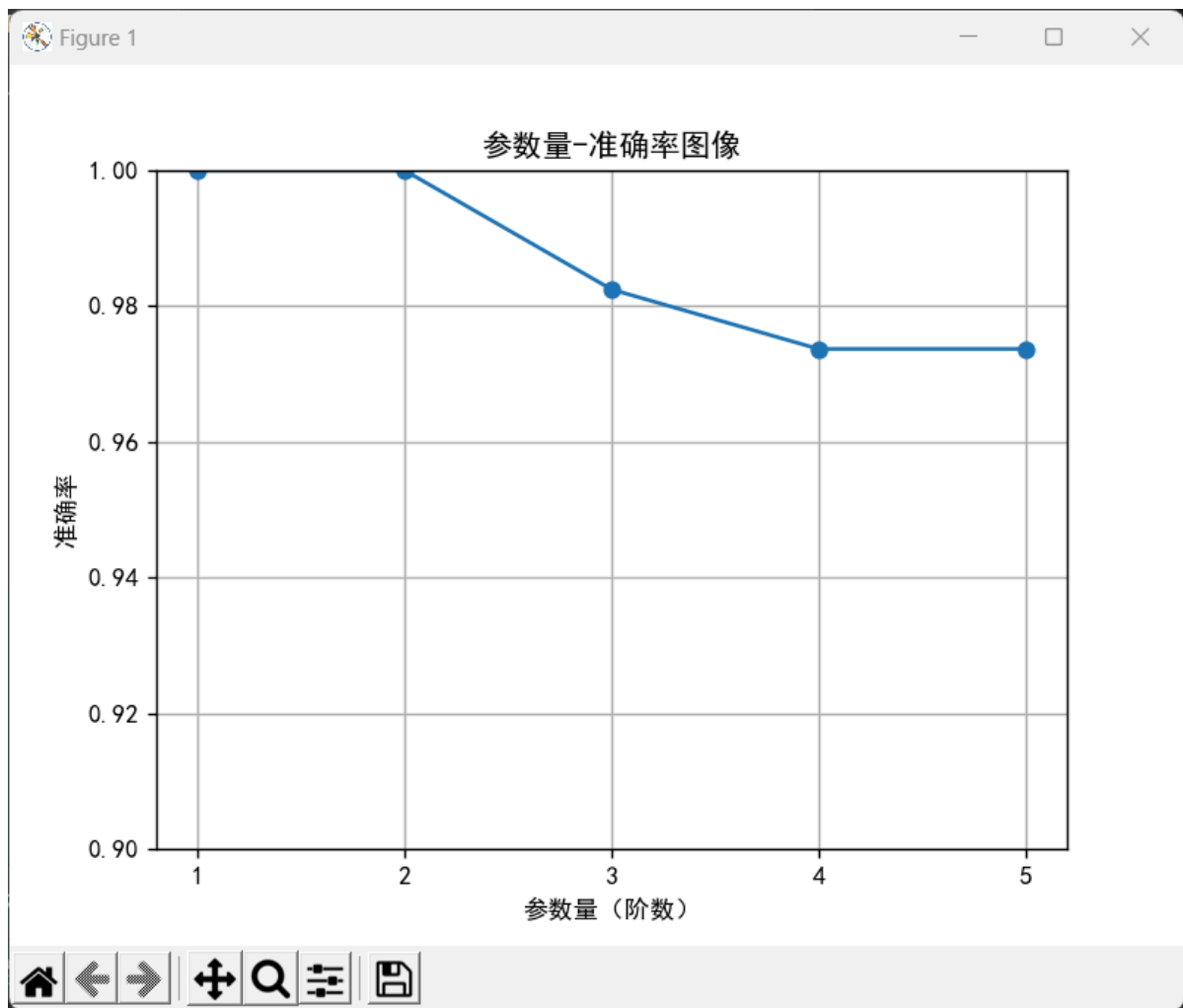
degree = 5:



使用所有特征进行训练，得到不同参数量下的结果

```
def train(degree,X_train,X_test,y_train,y_test):  
    # 生成多项式特征  
    poly = PolynomialFeatures(degree)  
    # 将输入数据转换为多项式特征  
    X_poly_train = poly.fit_transform(X_train)  
    # 使用在训练集上学到的特征组合和参数，将 X_test 转换为相同的多项式特征矩阵  
    X_poly_test = poly.transform(X_test)  
    # 模型实例化&训练  
    model = LogisticRegression()  
    model.fit(X_poly_train, y_train)  
  
    # 使用已训练的逻辑回归模型对测试数据进行分类  
    y_predict = model.predict(X_poly_test)  
  
    # 模型评估  
    accuracy = accuracy_score(y_test, y_predict)  
    return accuracy
```

不同参数量下的结果



如图可见，随着多项式阶数上升，准确率反而下降。

这是因为参数量太多时，模型和训练集上的数据过拟合，导致模型在测试集上性能下降。

SVM模型的一般理论

SVM的核心思想是通过优化一个凸二次规划问题，来在特征空间中找到最佳的超平面。它的基本模型是定义在特征空间上间隔最大的线性分类器。

SVM能够执行线性或非线性分类、回归，异常值检测任务。对于非线性可分的情况，SVM可以通过核函数将输入特征映射到高维空间，使得原本线性不可分的数据在高维空间中变得线性可分。

支持向量

支持向量是指在分类边界上，距离分类超平面最近的那些样本点。这些点决定了分类器的决策边界，因此对模型有很大的影响。

间隔最大化

间隔是指点到超平面的间隔。间隔最大化是指使得超平面和支持向量之间的距离尽可能的大。

间隔最大化分为软间隔最大化和硬间隔最大化。软间隔最大化的目标是在最大间隔宽度和减少间隔违例之间找到良好的平衡。硬间隔最大化则要求所有的实例都不在最大间隔之间，并且位于正确的一边。

核函数

核函数的作用是把低维空间中不可分的两类点变成高维空间中线性可分的。

不同核函数的SVM模型

线性核函数

```
TP: 50
TN: 86
FP: 5
FN: 2
Precision: 0.9524793388429753
Recall: 0.951048951048951
F-score: 0.95
Accuracy: 0.951048951048951
```

高斯核函数

```
[32 rows x 7 columns]
TP: 46
TN: 88
FP: 3
FN: 6
Precision: 0.937117593652548
Recall: 0.9370629370629371
F-score: 0.94
Accuracy: 0.9370629370629371
```

线性核函数的Precision, Recall, F-score, Accuracy都比高斯核函数的高，所以可以认为，使用这个模型在这个任务上，采用线性核函数时的性能更好。

SVM的训练过程

模型初始化方法

一般使用SVC函数进行模型的初始化。其中的参数有：

1. gamma: 核函数的系数，可以影响决策边界的形状。gamma的值有scale和auto两种，默认是auto。
2. C: 惩罚参数。C越大，对误分类的惩罚越大。
3. decision-function_shape: 决策参数类型。可以选择ovo(one vs one)和ovr(one vs rest)

4. kernel: 核函数类型。
5. degree: 多项式阶数。
6. coef0: 核函数中的独立项。对linear和rbf核函数无用。
7. shrinking: 是否采用启发式收缩方式。
8. tol: 停止训练的误差精度，默认 $1e^{-3}$ 。
9. cache_size: 训练需要的内存大小。
10. class_weight: 不同类别的权重。
11. max_iter: 最大迭代次数。
12. random_state: 数据洗牌时的种子值。

超参数选择

1. C: 一般C的范围在0.1-100之间，这里采用100。
2. decision_function_shape: 这里只有两类，使用ovo。
3. kernel: 分别使用线性和高斯核函数。

训练技巧

正则化：在SVM模型中正则化方法主要通过调整C参数来实现。分别尝试了默认值（C=1），10,30,50,70,100,150共七个C参数的值，最终C=100时准确率最高。

SMO：尝试使用序列最小优化算法。训练中遍历到每个样本时，计算样本预测值和误差。如果样本违反KKT条件，就要更新偏置，并把这个样本用作拉格朗日乘子。更新偏置时，再选择一个违反KKT条件的样本作为乘子，并计算它的预测值和误差。然后，优化两个乘子。接着，用优化后的乘子更新偏置。在实验中尝试使用了SMO算法优化SVM，但是模型不收敛，可能是算法实现有误。

实验结果分析与讨论

代码实现

```
predictor = svm.SVC(C=100.0, decision_function_shape='ovo', kernel='rbf')
# 进行训练
predictor.fit(x_train, y_train)
# 预测结果
result = predictor.predict(x_test)
# 进行评估
acc = accuracy_score(y_test, result)
cm = confusion_matrix(y_test, result)
TP = cm[1, 1] # 真正类
TN = cm[0, 0] # 真负类
FP = cm[0, 1] # 假正类
FN = cm[1, 0] # 假负类
precision = precision_score(y_test, result, average='weighted')
recall = recall_score(y_test, result, average='weighted')
```

评价指标

Precision: 在所有被模型预测为正类的样本中，实际为正类的比例。高斯核函数模型的Precision为0.937，线性核函数模型的Precision为0.952。

Recall: 在所有实际为正类的样本中，模型预测为正类的比例。高斯核函数的Recall为0.937，线性核函数的Recall为0.951。

F-score: 精确率和召回率的调和平均数。高斯核函数的F-score为0.94，线性核函数的F-score为0.95。

Accuracy: 模型正确预测的样本数占总样本数的比例。高斯核函数模型的Accuracy为0.937，线性核函数模型的Accuracy为0.951。

分析与讨论

SVM是一种线性分类器，主要用于进行中小规模的分类任务。在本次实验中，使用了线性核函数和高斯核函数的SVM模型进行任务。经过参数调整，线性核函数的SVM模型分类准确率在0.95，高斯核函数的SVM模型分类准确率在0.93。因此，在本次任务中，线性核函数的SVM模型性能要高于高斯核函数的模型。

此外，在训练模型时，通过调整参数实现了正则化方法。通过正则化方法，线性核函数的SVM模型分类准确率从0.93提升到了0.95，高斯核函数的SVM分类准确率从0.92提升到了0.937。

同时，尝试了使用SMO算法优化模型。